

GETTING STARTED

Quick Start

Generating a Card

Config Object

Branding Assets

Web Server

REST API

Template Versioning

Reference

Kaascan SDK · v1.0.0

MIT License · open source

Developer Integration Guide

Generate student ID cards from your app with a handful of clean calls. This guide covers the Python SDK, the REST web API, and the web server for template previews.

[Python · FastAPI · Open Source](#)

Quick Start

Install the dependencies and start the preview server in two commands.

```
pip install -r requirements.txt
# start the web UI (optional, for template previews)
python -m card_sdk.web_server
```

Note

The package lives in `sdk/card_sdk/`. Add the `sdk/` folder to your `PYTHONPATH` or install it so `import card_sdk` resolves to the repo copy.

Generating a Card

For a single card, `generate_card()` is all you need. It returns a status string and writes the finished PDF to the output folder.

```
example.py

from card_sdk.card import generate_card, image_file_to_base64

result = generate_card(
    template_name="BLUE GREEN STUDENT CARD",
    image_base64=image_file_to_base64("student.jpg"),
    student_name="Tuyishimire Fraterine Ely",
    student_class="Software Development",
    school_name="KANYINYA TSS 001",
    student_id="S-2026-001",
    data_qrcode="https://kaascan.com/verify/S-2026-001",
)
print(result) # status message + output path
```

Output goes to `output/<school_name>/<class>/final_student_card_<id>.pdf`.

Choose a template

```
Python

from card_sdk.template_manager import pull_template_options, pull_template_files

options = pull_template_options() # e.g. ["BLUE GREEN STUDENT CARD", "RUNO SCHOOL CARD", ...]
```



One call

`generate_card()` renders the card and writes the final PDF automatically.



Template list

`pull_template_options()` returns every installed template by name.



Extensible

Branding assets, colors and typography are all optional keyword arguments.

Config Object

Wrap your data in a `CardConfig`. It centralises the values and can build a pydantic `Student` model for the asynchronous batch flow.

```
config_usage.py

from card_sdk.card import CardConfig, image_file_to_base64
```

```
photo = image_file_to_base64("student.jpg")

cfg = CardConfig(
    photo=photo,                                # required: base64 student photo
    template_name="BLUE GREEN STUDENT CARD",
    student_name="Tuyishimire Fraterine Ely",
    student_class="Software Development",
    school_name="KANYINYA TSS 001",
    student_id="S-2026-001",
    data_qrcode="https://kaascan.com/verify/S-2026-001",
    # optional branding: school_logo, stamp, signature, barcode (all base64)
)

# Option A: render the card content dict
content = cfg.generate_card_content()  # renders card + writes PDF

# Option B: build a Student model for the async batch API
student = cfg.to_student()             # card_sdk.card_agent.Student instance
```

Branding Assets

Pass base64-encoded assets as optional keyword arguments. Every parameter has a sensible default, so integrations only need to send what they have.

```
branding.py

from card_sdk.card import generate_card, image_file_to_base64

result = generate_card(
    template_name="BLUE GREEN STUDENT CARD",
    image_base64=image_file_to_base64("student.jpg"),
    student_name="Alice Johnson",
    student_class="Grade 4",
    school_name="SUNRISE ACADEMY",
    student_id="A-001",
    data_qrcode="https://kaascan.com/verify/A-001",
    # ---- optional branding ----
    school_logo=image_file_to_base64("logo.png"),
    stamp_base64=image_file_to_base64("stamp.png"),
    student_barcode=image_file_to_base64("barcode.png"),
    signature_base64=image_file_to_base64("signature.png"),
    side_2_color="#00897B",
)
```

Image helpers

Helper	Description
<code>image_file_to_base64(path)</code>	Read a local image file and base64-encode it.

Helper	Description
<code>image_url_to_base64(url)</code>	Synchronously fetch a remote image and encode it.
<code>resize_image_base64(b64, w=100, h=100)</code>	Resize an encoded image for smaller cards.

Web Server

Start the FastAPI web UI from the terminal. This is what the terminal UI's "Start the web server to view templates?" prompt calls for you.

```
Terminal
python -m card_sdk.web_server # port 8000
python -m card_sdk.web_server --port 9000
python -m card_sdk.web_server --host 127.0.0.1
```

Or start it from code:

```
Python
from card_sdk.web_server import serve_web
serve_web(host="0.0.0.0", port=8000) # blocks until interrupted
```

The server binds to the local dev SDK automatically (it adds the repo to the path), so you always see the real templates.

REST API

Once the server is running, call the endpoints from any client (curl, JavaScript, Postman, ...).

Method	Endpoint	Purpose
GET	<code>/healthy/status</code>	Health check. → <code>{status, time_stamp}</code>
GET	<code>/card/templates</code>	List template names + preview paths. → <code>[{template_name, front_preview_path, back_preview_path}]</code>
GET	<code>/student/photo/<student_id></code>	Get a cached student photo (PNG).
GET	<code>/student/card/<front back>/<student_id></code>	Get a rendered card side as SVG.

```
JavaScript / fetch
```

```
const res = await fetch("http://localhost:8000/card/templates");
const templates = await res.json();
console.log(templates.map(t => t.template_name));
```

```
curl http://localhost:8000/card/templates
curl http://localhost:8000/student/card/front/S-2026-001
```

Tip

Interactive API docs are available at <http://localhost:8000/docs> (Swagger UI).

Template Versioning

Card designs live as SVG source files in `card_sdk/templates/templates_base/<template>/`. The generator reads a minified `.card.kaascan` file for each side, so applying a design update used to mean hand-collapsing the SVG and re-editing every card. The versioning CLI turns template design into simple, git-style commands.

When a designer ships a new look, drop `front.svg` / `back.svg` into the template folder and commit. The design is snapshotted, the `.card.kaascan` files are rebuilt so the generator picks it up immediately, and it becomes the active version. Old designs stay under `versions/` and can be restored at any time — no card rewriting needed.

```
python3 -m card_sdk.template_versioning build BANK_INSPIRE
python3 -m card_sdk.template_versioning commit BANK_INSPIRE --label spring-2026 --message "Spring rebrand"
python3 -m card_sdk.template_versioning list
python3 -m card_sdk.template_versioning current BANK_INSPIRE
python3 -m card_sdk.template_versioning rollback BANK_INSPIRE --label v1
```

Folder layout

```
templates_base/BANK_INSPIRE/
├── front.svg · back.svg                # working design (source of truth)
├── front.card.kaascan · back.card.kaascan # rebuilt from the svg – generator reads these
└── versions/
    ├── .current                        # active label
    ├── v1/ (front.svg, back.svg, meta.json) # one snapshot per label, like a branch
    └── spring-2026/ (front.svg, back.svg, meta.json)
```

Commands

Command	Purpose
<code>build <template></code>	Rebuild the <code>.card.kaascan</code> files from the SVGs in the folder — e.g. right after hand-editing a design. Warns about missing standard placeholders.
<code>commit <template> --label <name> [--message "..."]</code>	Snapshot the current design under <code>versions/<label></code> , sync the <code>.kaascan</code> cards and mark it active. Reusing a label needs <code>--force</code> .
<code>list [<template>]</code>	All committed versions (label, date, message); <code>*</code> marks the active one. Without a template it covers every installed template.
<code>current <template></code>	Show the active version label.
<code>rollback <template> --label <name></code>	Restore the working SVG + <code>.kaascan</code> from a saved snapshot and make that design active again.

Workflow

Designer drops `front.svg` + `back.svg` → `commit --label spring-2026` → cards render with the new design immediately; the previous design is still one `rollback --label v1` away.

Keep the placeholders

Card data is filled through `{...}` placeholders inside the SVG (`{name}` , `{student_code}` , `{student_class}` , `{school_name}` , `{student_id}` , `{student_photo}` , `{student_barcode}` , `{valid_thru}` , `{school_type}`). `build` / `commit` warn when a freshly-dropped design is missing one, so card fields never silently stay empty.

Reference

generate_card()

Signature

copy

```
generate_card(  
    template_name: str,  
    image_base64: str,           # student photo (base64)  
    student_name: str,  
    student_class: str,  
    school_name: str,  
    student_id: str,  
    student_code="",           # card number, e.g. "5298 7601 2345 6789"  
    valid_thru="09/30",        # shown under "VALID THRU"  
    school_type="HIGH SCHOOL", # school category  
    data_qrcode: str = "",     # payload for the QR code  
    side_2_color="#00897B",  
    school_logo="",  
    stamp_base64="",
```

```

student_barcode="",
signature_base64="",
snFontSize=2.4932, snx=-10.523738, sny=-0.8769781,
) -> str

```

Public entry points

Symbol	From	Notes
<code>generate_card</code>	<code>card_sdk.card</code>	Render a single card (returns status string).
<code>save_card(content, id, class, school)</code>	<code>card_sdk.card</code>	Convert card SVG content to the final PDF.
<code>CardConfig</code>	<code>card_sdk.card</code>	Config object → <code>generate_card_content()</code> / <code>to_student()</code> .
<code>Card</code>	<code>card_sdk.card_agent</code>	Async batch entry point — <code>await Card.agent(data=student)</code> .
<code>Student</code>	<code>card_sdk.card_agent</code>	Pydantic model (photo, name, Class, school_name, data_qrcode, student_id, template_name, + student_code, valid_thru, school_type).
<code>pull_template_options</code>	<code>card_sdk.template_manager</code>	List of template names.
<code>pull_template_files</code>	<code>card_sdk.template_manager</code>	Detailed template file list (front/back paths).
<code>serve_web</code>	<code>card_sdk.web_server</code>	Start the FastAPI web UI.
<code>template_versioning</code>	<code>card_sdk.template_versioning</code>	Git-style design versions — <code>python3 -m card_sdk.template_versioning list</code> .

Tip

The effective card *content* (SVG) lives in the returned value of `generate_card()` & `CardConfig.generate_card_content()`; the final PDF is produced by `save_card()` / the card agent. For most integrations, calling `generate_card()` is enough — the PDF is written automatically.